

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

Escuela Profesional de Ingeniería Informática y de Sistemas

Curso: IF614 – Ingeniería de Software I

SISTEMA INTELIGENTE DE RECOLECCIÓN DE RESIDUOS SÓLIDOS SEGREGADOS – CUSCO

Entrega N.º 2 – Documentación y sistema funcional

Metodología: SCRUM | Atributo del graduado: AG-C01

Semestre 2026-I

Integrantes:

Luicho Quispe, Jhoel Alex (224871) | Luna Ccapa, Carlos Willian (210178)
Puma Condori, Richard Braulio (161809) | Huaracallo Arenas, Lino Zeynt (204798)

Cusco – Perú, 2026

Contenido

- ① **Problema y objetivos** del proyecto
- ② **Alcance** y público objetivo
- ③ **Metodología SCRUM**: roles y Product Backlog
- ④ **Sprints** realizados
- ⑤ **Arquitectura y tecnologías**
- ⑥ **Sistema construido**: backend, app móvil y dashboard web
- ⑦ **Despliegue** en la nube y APK
- ⑧ **Diagramas** (UML / Entidad-Relación)
- ⑨ **Resultados y conclusiones**

1 — Problema y Objetivos

Problema

En el Cusco la recolección de residuos presenta:

- Rutas no optimizadas, sin trazabilidad
- Escasa comunicación con la población
- Baja cultura de segregación
- Ausencia de reportes para decidir

Objetivo general

Diseñar e implementar un sistema inteligente de gestión de residuos sólidos segregados que optimice la recolección, mejore la comunicación con la ciudadanía y genere información para la toma de decisiones, aplicando SCRUM.

Objetivos específicos

Gestión de usuarios y zonas · catálogo y guía de segregación · monitoreo de rutas con ubicación · app móvil ciudadana · sistema de alertas · reportes y analítica.

2 — Alcance y Público Objetivo

Alcance

- Administración de usuarios y zonas
- Catálogo de residuos y segregación
- Monitoreo de rutas
- App móvil ciudadana
- Alertas y reportes

Fuera de alcance: hardware IoT, pasarelas de pago, sistemas tributarios.

Público objetivo

- **Ciudadanos:** rutas, alertas, reportar incidencias, segregar
- **Operadores:** ejecutar rutas, enviar ubicación
- **Administradores:** zonas, rutas, usuarios, reportes

3 — Metodología SCRUM: Roles

Equipo Scrum

Rol	Integrante	Función
Scrum Master	Huaracallo Arenas, Lino Zeynt	Facilita el proceso, remueve impedimentos
Product Owner	Luicho Quispe, Jhoel Alex	Prioriza el backlog, criterios de aceptación
Developer (Full Stack)	Luna Ccapa, Carlos Willian	Backend, app móvil y dashboard
Developer (Full Stack)	Puma Condori, Richard Braulio	Backend, app móvil y dashboard
Developer (Full Stack)	Luicho Quispe, Jhoel Alex	Integración y despliegue

Todos los integrantes participan como desarrolladores full stack; Lino asume además el rol de Scrum Master.

Herramientas de colaboración

GitHub (repositorio y ramas) · Render (despliegue) · MongoDB Atlas (BD) · Expo (app móvil).

3 — Product Backlog (Requisitos)

Requisitos Funcionales

ID	Requisito	Est.
RF-01	Registro de ciudadanos	✓
RF-02	Autenticación (JWT)	✓
RF-03/04	Zonas + asignación (GPS)	✓
RF-05/06	Tipos y clasificación de residuos	✓
RF-07/08/09	Rutas + ubicación + gestión	✓
RF-10	Consulta de rutas (móvil)	✓
RF-11	Reporte de incidencias (GPS)	✓
RF-14	Reporte de residuos por zona	✓
RF-12/13/15/16	Alertas y reportes avanzados	~

✓ completado ~ parcial

Requisitos No Funcionales

- Seguridad: bcrypt + JWT
- Disponibilidad: nube 24/7
- Usabilidad: interfaz simple
- Rendimiento: < 3 s
- Portabilidad: APK Android
- Privacidad: Ley 29733

4 — Sprints Realizados

3 Sprints · 2 semanas cada uno

Sprint	Objetivo	Entregables
1	Autenticación y gestión de usuarios/zonas	API auth (JWT), modelos, zonas, detección por GPS
2	Núcleo operativo + app móvil	Rutas + ubicación, residuos, incidencias, pantallas móviles
3	Alertas, reportes, web y despliegue	Dashboard web, reportes, despliegue en la nube, APK

Ramas por historia (GitHub)

`feature/HU01-registro-ciudadanos ... feature/HU08-dashboard-admin`

Flujo: rama por HU → commit → Pull Request → merge a main → despliegue.

5 — Arquitectura y Tecnologías

Arquitectura (cliente–servidor)

- **Cientes:** app móvil (Android) + dashboard web
- **Servidor:** API REST (Node/Express) en Render
- **Datos:** MongoDB Atlas (nube)
- Comunicación HTTPS/JSON, autenticación JWT

Tecnologías

- Backend: **Node.js + Express**
- Base de datos: **MongoDB** (Mongoose)
- App: **Expo / React Native** (TypeScript)
- Web: HTML + JS (servido por Express)
- Mapas: OpenStreetMap / Leaflet
- Integración: API RENIEC (DNI)

Flujo

App móvil / Web → API REST (Render) → MongoDB Atlas

6 — Sistema: Backend (API REST)

Modelos de datos

- Usuario (roles, zona, ubicación)
- Zona (polígono geográfico)
- Ruta (estado, ubicación en vivo)
- Incidente, Residuo, Alerta

Seguridad

Contraseñas con **bcrypt** · sesión con **JWT** · control de acceso por rol.

Endpoints principales

- `/auth/register`, `/auth/login`
- `/zonas`, `/zonas/detect` (GPS)
- `/rutas`, `/rutas/:id/ubicacion`
- `/incidentes`, `/residuos`
- `/usuarios`, `/alertas`, `/dni/:dni`

Detección de zona

Consulta geoespacial **punto-en-polígono** para asignar la zona del ciudadano por su ubicación.

6 — Sistema: App Móvil y Dashboard Web

App móvil (Android / APK)

- Login y registro (mapa + GPS)
- Autocompletar datos por **DNI** (RENIEC)
- Inicio: rutas y estado
- **Mapa: camiones en vivo**
- Reporte de incidencias con foto/ubicación
- Guía de segregación · Perfil

Dashboard web (administración)

- Resumen / indicadores
- Gestión de usuarios y zonas
- Gestión de rutas y estados
- Gestión de incidencias
- Catálogo de residuos

Servido por el mismo backend (misma URL).

[Insertar capturas reales de la app y el dashboard]

7 — Despliegue en la Nube y APK

Despliegue

- **Backend + Web:** Render (en línea 24/7)
- **Base de datos:** MongoDB Atlas
- **Repositorio:** GitHub (main + ramas)
- **App:** APK Android instalable

Funciona desde cualquier red

El sistema deja de depender de una PC local: la app y el dashboard se conectan a un backend en la nube con URL permanente.

Estado de la Entrega 2

Sistema **funcional** (supera el 50 % requerido): backend + app + dashboard desplegados.
Documentación: Capítulo I, II y bosquejo del III.

8 — Diagramas (UML / Entidad-Relación)

Diagramas elaborados

- **Casos de uso** — actores: ciudadano, operador, administrador
- **Clases / dominio** — Usuario, Zona, Ruta, Incidente, Residuo, Alerta
- **Entidad-Relación** — colecciones y relaciones en MongoDB
- **Secuencia** — login y reporte de incidencia
- **Componentes** — clientes, API y base de datos

[Insertar aquí los diagramas — disponibles en docs/DIAGRAMAS.md]

9 — Resultados

Logros

- Sistema de 3 capas funcional y desplegado
- 11 de 16 requisitos funcionales completos
- App móvil instalable (APK)
- Integración con API RENIEC (DNI)

Impacto (AG-C01)

- **Ambiental:** promueve la segregación
- **Social:** participación ciudadana
- **Legal:** Ley 29733, Ley 27314, NTP 900.058

Pendiente

Notificaciones push de cercanía/retraso (RF-12/13) y reportes avanzados (RF-15/16) — previstos para la siguiente etapa.

Conclusiones

① Sistema funcional desplegado

Backend (Node/Express + MongoDB), app móvil (Expo) y dashboard web, en la nube — supera el 50 % requerido para la Entrega 2.

② SCRUM aplicado

Product Backlog priorizado, 3 sprints, ramas por historia de usuario y trabajo colaborativo en GitHub.

③ Cobertura de requisitos

11/16 RF completos; los módulos esenciales (usuarios, zonas, rutas, residuos, incidencias) operativos.

④ Responsabilidad ética y legal

Cifrado de datos, manejo responsable de información personal (Ley 29733) y alineación con normas ambientales.

⑤ Próximos pasos

Notificaciones push, reportes avanzados y refinamiento del dashboard para la entrega final.

Gracias por su atención
